МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №2
по курсу «Программирование графических процессоров»

Обработка изображений на GPU. Фильтры

Выполнил: Ефимов Сергей Юрьевич
Группа: М80-403Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Научиться использовать GPU для обработки изображений. Использование текстурной памяти и двухмерной сетки потоков. Изображение является бинарным файлом, со следующей структурой:

width(w)	height(h)	r g b a	...	r g b a
4 байта, int	4 байта, int	4 байта, значение пикселя [1,1]	...	4 байта значение пикселя [w,h]

В первых восьми байтах записывается размер изображения, далее построчно все значения пикселей, где

- r -- красная составляющая цвета пикселя
- g -- зеленая составляющая цвета пикселя
- b -- синяя составляющая цвета пикселя
- a -- значение альфа-канала пикселя

Пример картинки размером 2 на 2, синего цвета, в шестнадцатеричной записи:
02000000 02000000 0000FF00 0000FF00 0000FF00 0000FF00
Необходимо написать конвертер на любом языке программирования для работы с вышеописанным форматом.
В данной лабораторной работе используются только цветовые составляющие изображения (r g b), альфа-канал не учитывается. При расчетах значений допускается ошибка в ±1. Ограничение: w < 2^16 и h < 2^16. Для любых индексов i и j, координаты пикселя [ip, jp] будут определятся следующим образом:
ip := max(min(i, h), 1)
jp := max(min(i, w), 1)
Вариант 5. Выделение контуров. Метод Робертса.
Входные данные. На первой строке задается путь к исходному изображению, на второй, путь к конечному изображению. w*h <= 10^8

Программное и аппаратное обеспечение

Видеокарта

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 615.71.09              KMD Version: 615.71.09      CUDA UMD Version: 13.4
|
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================
+======================|
|   0  NVIDIA GeForce GTX 1660 ...    Off |   00000000:01:00.0  On |                  N/A |
| 43%   31C    P8             11W /  125W |     557MiB /   6144MiB |     16%      Default |
```

```
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+
Device 0
Compute capability      : 7.5
Name                    : NVIDIA GeForce GTX 1660 SUPER
Total Global Memory     : 6034096128
Shared memory per block : 49152
Registers per block     : 65536
Warp size               : 32
Max threads per block   : (1024, 1024, 64)
Max block               : (2147483647, 65535, 65535)
Total constant memory   : 65536
Multiprocessors count   : 22

Процессор
Архитектура:                x86_64
  CPU op-mode(s):           32-bit, 64-bit
  Address sizes:            39 bits physical, 48 bits virtual
  Порядок байт:             Little Endian
CPU(s):                     12
  On-line CPU(s) list:      0-11
ID прроизводителя:          GenuineIntel
  Имя модели:               11th Gen Intel(R) Core(TM) i5-11500 @ 2.70GHz
    Семейство ЦПУ:          6
    Модель:                 167
    Thread(s) per core:     2
    Ядер на сокет:          6
    Сокетов:                1
    Степпинг:               1
    Microcode version:      0x66
    CPU(s) scaling MHz:     92%
    CPU max MHz:            4600,0000
    CPU min MHz:            800,0000
    BogoMIPS:               5424,00

Оперативная память
               total        used        free      shared  buff/cache   available
Mem:            15Gi       7,0Gi       1,4Gi       237Mi       7,8Gi       8,5Gi
Swap:           15Gi       6,2Mi        15Gi

Жесткий диск
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda           8:0    0 931,5G  0 disk
└─sda1        8:1    0   931G  0 part
```

```
sr0          11:0    1  1024M  0 rom
zram0        253:0    0  15,4G  0 disk [SWAP]
nvme0n1      259:0    0   1,9T  0 disk
├─nvme0n1p1 259:1    0     4G  0 part /boot
└─nvme0n1p2 259:2    0   1,9T  0 part /var/tmp
                                /var/log
                                /var/cache
                                /home
                                /root
                                /srv
                                /
```

Система
Linux cachyos-x8664 7.2.6-1-cachyos #1 SMP PREEMPT_DYNAMIC Thu, 17 Sep 2026 15:20:14 +0000 x86_64 GNU/Linux

IDE
CLion от JetBrains

Компилятор:
Для локального тестирования использовался CMake со следующей конфигурацией:

```
cmake_minimum_required(VERSION 4.3)
set(CMAKE_CUDA_COMPILER /opt/cuda/bin/nvcc)
set(CUDAToolkit_ROOT /opt/cuda)
project(pgp_tpp LANGUAGES CXX CUDA)
set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CUDA_STANDARD 14)
set(CMAKE_CUDA_STANDARD_REQUIRED ON)
```

При отправке использовался makefile

```
NVCC = /usr/bin/nvcc
CFLAGS = --std=c++14
SOURCES = lab2.cu
BIN = lab2
all:
	$(NVCC) $(CFLAGS) -o $(BIN) $(SOURCES)
```

Метод решения

Для реализации метода Робертса с использованием CUDA нужна функция с __global__ для запуска обработки изображения на GPU. В функции происходит выделение границ изображения с распределением потоков по пикселям. Каждый поток считывает из текстурной памяти четыре соседних пикселя, переводит их цвета в яркость и вычисляет

модуль градиента по разностям яркостей вдоль двух диагоналей. Полученное значение ограничивается диапазоном от 0 до 255 и записывается в результирующее изображение. В главной функции main происходит считывание изображения из файла, выделение памяти на GPU и перенос данных из CPU в GPU, после чего создается текстурный объект и запускается функция обработки. Полученный результат переносится из GPU в CPU и сохраняется в выходной файл. После этого программа удаляет текстурный объект и освобождает память на GPU.

Описание программы

Функция robertsKernel необходима для выделения границ изображения методом Робертса с использованием CUDA. Внутри используются blockIdx (номер блока), blockDim (количество потоков внутри блока) и threadIdx (номер потока) для определения координат x и y пикселя, обрабатываемого каждым потоком. Если координаты выходят за размеры изображения, поток завершает работу. С помощью tex2D из текстурной памяти считываются четыре соседних пикселя, образующих область 2×2. Функция grayscale вычисляет яркость каждого пикселя по его цветовым компонентам. Далее вычисляются разности яркостей вдоль двух диагоналей и модуль градиента. Полученное значение ограничивается сверху числом 255 и записывается в три цветовых канала результирующего пикселя. В главной функции main считываются имена входного и выходного файлов, затем функция readImage загружает размеры и пиксели изображения. С помощью cudaMallocArray выделяется память на GPU для исходного изображения, а cudaMemcpy2DToArray переносит в нее данные из CPU. Далее создается текстурный объект с ограничением координат по краям изображения, а с помощью cudaMalloc выделяется память под результат. Задаются блоки размером 16×16 потоков и рассчитывается количество блоков, необходимое для обработки всего изображения. После запуска robertsKernel программа ожидает завершения всех потоков с помощью cudaDeviceSynchronize(). Затем cudaMemcpy переносит результат из GPU в CPU, а функция writeImage сохраняет его в выходной файл. В конце работы текстурный объект удаляется с помощью cudaDestroyTextureObject, а выделенная память освобождается с помощью cudaFree и cudaFreeArray.

Результаты

Время работы ядер с различными конфигурациями

Image: 256 x 256

Blocks	Threads	Time (ms)	Result
1	32	0.930509	OK
1	64	0.469469	OK
1	128	0.246896	OK
1	256	0.145088	OK
1	512	0.145386	OK
1	1024	0.151174	OK
2	32	0.479053	OK
2	64	0.245914	OK

2	128	0.133923	OK
2	256	0.147254	OK
2	512	0.145690	OK
2	1024	0.151936	OK
4	32	0.245747	OK
4	64	0.134771	OK
4	128	0.136342	OK
4	256	0.145862	OK
4	512	0.145462	OK
4	1024	0.149754	OK
8	32	0.078867	OK
8	64	0.078698	OK
8	128	0.078262	OK
8	256	0.084752	OK
8	512	0.092080	OK
8	1024	0.104995	OK
16	32	0.077834	OK
16	64	0.078534	OK
16	128	0.078096	OK
16	256	0.085302	OK
16	512	0.085680	OK
16	1024	0.090570	OK
32	32	0.078870	OK
32	64	0.077930	OK
32	128	0.078621	OK
32	256	0.084477	OK
32	512	0.087101	OK
32	1024	0.087939	OK
64	32	0.079462	OK
64	64	0.078112	OK
64	128	0.086195	OK
64	256	0.085574	OK
64	512	0.088374	OK
64	1024	0.087859	OK
128	**32**	**0.077962**	**OK**
128	64	0.087050	OK
128	128	0.087379	OK
128	256	0.085562	OK
128	512	0.089283	OK
128	1024	0.093821	OK
256	32	0.078166	OK
256	64	0.086592	OK
256	128	0.090019	OK
256	256	0.090349	OK
256	512	0.092403	OK
256	1024	0.159725	OK
512	32	0.079981	OK
512	64	0.086650	OK

512	128	0.096912	OK
512	256	0.094310	OK
512	512	0.159747	OK
512	1024	0.316099	OK
1024	32	0.079530	OK
1024	64	0.089994	OK
1024	128	0.096234	OK
1024	256	0.156272	OK
1024	512	0.308970	OK
1024	1024	0.620285	OK

Image: 4096 x 4096

Blocks	Threads	Time (ms)	Result
1	32	179.531647	OK
1	64	88.717751	OK
1	128	44.411919	OK
1	256	28.186056	OK
1	512	15.315523	OK
1	1024	10.525367	OK
2	32	89.384552	OK
2	64	45.121677	OK
2	128	22.721395	OK
2	256	13.845737	OK
2	512	8.018381	OK
2	1024	5.473478	OK
4	32	39.869423	OK
4	64	22.855932	OK
4	128	12.288018	OK
4	256	6.884254	OK
4	512	4.213267	OK
4	1024	2.824666	OK
8	32	19.622782	OK
8	64	10.544560	OK
8	128	5.605990	OK
8	256	3.583008	OK
8	512	2.305587	OK
8	1024	4.660006	OK
16	32	15.723117	OK
16	64	6.713943	OK
16	128	3.785562	OK
16	256	2.414515	OK
16	512	2.971728	OK
16	1024	3.965603	OK
32	32	17.329782	OK
32	64	3.954650	OK
32	128	2.226051	OK
32	**256**	**2.193572**	**OK**
32	512	2.803706	OK

32	1024	3.638809	OK
64	32	5.219376	OK
64	64	6.517602	OK
64	128	7.680036	OK
64	256	9.689543	OK
64	512	10.324290	OK
64	1024	4.261146	OK
128	32	2.605536	OK
128	64	2.641069	OK
128	128	2.830947	OK
128	256	3.211133	OK
128	512	10.554515	OK
128	1024	11.690191	OK
256	32	2.745879	OK
256	64	2.771990	OK
256	128	2.733389	OK
256	256	3.381660	OK
256	512	4.402189	OK
256	1024	9.936129	OK
512	32	6.693027	OK
512	64	8.465532	OK
512	128	6.734992	OK
512	256	4.201411	OK
512	512	5.738704	OK
512	1024	11.219881	OK
1024	32	2.188112	OK
1024	64	2.241613	OK
1024	128	2.290653	OK
1024	256	3.343101	OK
1024	512	6.413300	OK
1024	1024	13.742330	OK

Сравнение gpu и cpu

Image: 1 x 1
CPU time: 0.000184 ms
GPU kernel time: 0.009581 ms
Speedup: 0.019216
Grid: 1 x 1
Block: 16 x 16
Results: OK

Image: 4096 x 4096
CPU time: 578.610973 ms
GPU kernel time: 13.892390 ms
Speedup: 41.649490
Grid: 256 x 256

Block: 16 x 16
Results: OK

Выводы

Данная лабораторная работа познакомила нас с обработкой изображений на GPU с использованием CUDA и текстурной памяти. В ходе работы был реализован метод Робертса для выделения границ изображения. Каждый поток обрабатывал отдельный пиксель, вычисляя модуль градиента по яркостям соседних пикселей. Использование текстурной памяти позволило удобно организовать доступ к пикселям и обработку краев изображения. Основное внимание при реализации потребовалось уделить распределению потоков по двумерному изображению и настройке текстурного объекта. Работа позволила закрепить навыки выделения памяти на GPU, передачи данных между CPU и GPU и запуска CUDA-ядер. Эффективность такого подхода зависит от размера изображения, поскольку при небольшом объеме данных затраты на передачу данных и запуск ядра могут превышать выигрыш от параллельной обработки, однако в реальных ситуациях GPU с большим отрывом выигрывает CPU